

# Contents

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Keep Me Ready — Developer Onboarding Guide v1</b>  | <b>2</b> |
| Table of Contents . . . . .                           | 2        |
| 1. Before You Start . . . . .                         | 2        |
| 2. Prerequisites . . . . .                            | 3        |
| 3. Clone the Repository . . . . .                     | 3        |
| 4. Install Dependencies . . . . .                     | 4        |
| 5. Set Up Supabase . . . . .                          | 4        |
| Create a new project . . . . .                        | 4        |
| Collect your credentials . . . . .                    | 4        |
| 6. Configure Environment Variables . . . . .          | 4        |
| 7. Run the Database Migrations . . . . .              | 5        |
| Migration 001 — run first . . . . .                   | 5        |
| Migration 002 — run second . . . . .                  | 5        |
| Verify . . . . .                                      | 5        |
| 8. Configure Supabase Auth . . . . .                  | 6        |
| 9. Run the Development Server . . . . .               | 6        |
| 10. Smoke Test the App . . . . .                      | 6        |
| Anonymous drill flow . . . . .                        | 7        |
| Dashboard (anonymous) . . . . .                       | 7        |
| Authentication . . . . .                              | 7        |
| Admin dashboard . . . . .                             | 7        |
| Sign out . . . . .                                    | 7        |
| 11. Project Orientation . . . . .                     | 7        |
| The files you will touch most often . . . . .         | 8        |
| The dual data path . . . . .                          | 8        |
| The drill state machine . . . . .                     | 8        |
| The question bank shape . . . . .                     | 9        |
| 12. Next.js 16 — Breaking Changes to Know . . . . .   | 9        |
| Middleware is proxy.ts, not middleware.ts . . . . .   | 9        |
| Route params are async (Promise) . . . . .            | 10       |
| cookies() is async . . . . .                          | 10       |
| Tailwind v4 — no @tailwind directives . . . . .       | 10       |
| No separate eslint.config.js setup needed . . . . .   | 11       |
| 13. Working in This Codebase . . . . .                | 11       |
| Before starting any task . . . . .                    | 11       |
| Running a build before pushing . . . . .              | 11       |
| Linting . . . . .                                     | 11       |
| Adding questions to the question bank . . . . .       | 11       |
| Adding a new topic category . . . . .                 | 11       |
| Changing the database schema . . . . .                | 12       |
| 14. Deployment . . . . .                              | 12       |
| First-time setup . . . . .                            | 12       |
| After deploying, update Supabase Auth URLs . . . . .  | 12       |
| Triggering a redeploy without a code change . . . . . | 13       |
| Checking function logs . . . . .                      | 13       |
| 15. Common Issues and Fixes . . . . .                 | 13       |

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Magic link arrives but clicking it gives an error . . . . .        | 13 |
| /admin redirects to /auth/login even when signed in . . . . .      | 13 |
| /admin shows “Access Denied” with your email . . . . .             | 13 |
| Dashboard shows no data after completing a drill . . . . .         | 14 |
| Build fails with “Module not found” for Supabase imports . . . . . | 14 |
| Tailwind styles not applying . . . . .                             | 14 |
| cookies() throws “was called outside a request scope” . . . . .    | 14 |

## Keep Me Ready — Developer Onboarding Guide v1

**Prepared:** June 2026

**Version:** 1.0

**Folder:** docs/engineering/

**Related documents:** [Founder Technical Blueprint v1](#) · [PRD v1](#)

---

### Table of Contents

1. Before You Start
  2. Prerequisites
  3. Clone the Repository
  4. Install Dependencies
  5. Set Up Supabase
  6. Configure Environment Variables
  7. Run the Database Migrations
  8. Configure Supabase Auth
  9. Run the Development Server
  10. Smoke Test the App
  11. Project Orientation
  12. Next.js 16 — Breaking Changes to Know
  13. Working in This Codebase
  14. Deployment
  15. Common Issues and Fixes
- 

### 1. Before You Start

Read the following documents before touching any code. This is not optional — the decisions encoded in this codebase only make sense in context, and working without that context leads to mistakes that are hard to undo.

| Document                          | Time    | What you will learn                                           |
|-----------------------------------|---------|---------------------------------------------------------------|
| <a href="#">Founder Vision v1</a> | ~15 min | Why this product exists, who it serves, what it will never do |

| Document                               | Time    | What you will learn                                                        |
|----------------------------------------|---------|----------------------------------------------------------------------------|
| <a href="#">PRD v1</a>                 | ~30 min | What is built, what is planned, acceptance criteria, risks                 |
| <a href="#">Technical Blueprint v1</a> | ~30 min | Architecture, every file explained, auth flow, database schema, deployment |

After reading those three documents, this onboarding guide will take you the rest of the way to a running local environment.

## 2. Prerequisites

You need the following installed before cloning the repository.

| Tool           | Required version   | Check                       | Install                                       |
|----------------|--------------------|-----------------------------|-----------------------------------------------|
| <b>Node.js</b> | 20 or higher       | <code>node --version</code> | <a href="https://nodejs.org">nodejs.org</a>   |
| <b>npm</b>     | 9 or higher        | <code>npm --version</code>  | Bundled with Node.js                          |
| <b>Git</b>     | Any recent version | <code>git --version</code>  | <a href="https://git-scm.com">git-scm.com</a> |

You also need:

- A **Supabase account** — free tier is sufficient. Sign up at [supabase.com](https://supabase.com).
- A **GitHub account** with access to the `tshatiqua-bit/keep-me-ready` repository.
- An **SSH key** configured for GitHub (the remote uses SSH, not HTTPS).

To verify your SSH key works with GitHub:

```
ssh -T git@github.com
# Expected: Hi tshatiqua-bit! You've successfully authenticated...
```

## 3. Clone the Repository

```
git clone git@github.com:tshatiqua-bit/keep-me-ready.git
cd keep-me-ready
```

Confirm you are on the main branch:

```
git branch
# * main
```

## 4. Install Dependencies

```
npm install
```

This installs all dependencies listed in `package.json`, including:

- next 16.2.9 with React 19
- `@supabase/supabase-js` and `@supabase/ssr` for auth and database
- `tailwindcss` v4 and its PostCSS plugin
- TypeScript 5 and all type definitions

**Do not use `npm install --legacy-peer-deps` or `force-install`.** If you encounter peer dependency errors, check whether your Node version meets the requirement in Section 2 before trying anything else.

---

## 5. Set Up Supabase

You need a dedicated Supabase project for local development. Do not share credentials with the production project.

### Create a new project

1. Log in at [supabase.com](https://supabase.com) and click **New project**
2. Choose an organisation, enter a project name (e.g. `keep-me-ready-dev`), set a database password, and select a region close to you
3. Wait for the project to finish provisioning (about 60 seconds)

### Collect your credentials

You need four values from your project dashboard.

**Project URL and anon key** — go to **Settings → Data API**:

- `NEXT_PUBLIC_SUPABASE_URL` — the Project URL (format: `https://xxxx.supabase.co`)
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` — the anon public key

**Service role key** — on the same page, scroll down to find:

- `SUPABASE_SERVICE_ROLE_KEY` — the `service_role` key (keep this strictly server-side; it bypasses all RLS policies)

Keep this browser tab open — you will need these values in the next step.

---

## 6. Configure Environment Variables

Copy the example env file:

```
cp .env.local.example .env.local
```

Open `.env.local` and fill in your values:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key-here
ADMIN_EMAIL=you@example.com
```

**ADMIN\_EMAIL** is the email address of the user who will have access to the /admin instructor dashboard. Set it to any email you control — you will use it to sign in and verify the admin route.

`.env.local` is gitignored and must never be committed. The `.env.local.example` file (which is committed) documents the shape without any real values.

`NEXT_PUBLIC_` prefixed variables are embedded in the browser bundle at build time. The other two are server-side only and are never exposed to the client.

---

## 7. Run the Database Migrations

The app requires two SQL migrations to be applied to your Supabase project. Both must be run manually in the Supabase SQL Editor — there is no CLI migration runner in this project.

Go to your Supabase project dashboard → **SQL Editor** → **New query**.

### Migration 001 — run first

Copy and paste the entire contents of `supabase/migrations/001_init.sql` and click **Run**.

This creates the `drill_sessions` table with Row Level Security enabled, and two RLS policies allowing users to read and write only their own rows.

### Migration 002 — run second

Copy and paste the entire contents of `supabase/migrations/002_analytics.sql` and click **Run**.

This adds `category` and `started_at` columns to `drill_sessions`, creates the `drill_question_answers` table with its own RLS policies, and adds performance indexes.

**Order matters.** Migration 002 references the table created by 001. Running them out of order will produce an error.

Both migrations use `IF NOT EXISTS` guards and are safe to re-run if needed.

### Verify

After running both migrations, go to **Table Editor** in Supabase and confirm that `drill_sessions` and `drill_question_answers` both appear.

## 8. Configure Supabase Auth

The app uses email magic links. You need to tell Supabase where to redirect users after they click a link.

1. In your Supabase project, go to **Authentication → URL Configuration**
2. Set **Site URL** to `http://localhost:3000`
3. Under **Redirect URLs**, add `http://localhost:3000/auth/callback`
4. Click **Save**

You will update these URLs to the production Vercel URL when deploying (see Section 14).

**Email rate limits:** Supabase's free tier allows 3 auth emails per hour. If you are testing the magic link flow repeatedly, you may hit this limit. Use the same email address across tests to minimise requests, or use the **Supabase Auth admin UI** to sign in directly without email.

---

## 9. Run the Development Server

```
npm run dev
```

The app starts at <http://localhost:3000>.

Next.js 16 uses Turbopack by default in development. The first cold start takes a few seconds; subsequent page navigations are near-instant.

To run a production build locally (useful for catching build-time errors before pushing):

```
npm run build && npm run start
```

A clean build with no errors produces output similar to:

| Route (app)        | Size   | First Load JS |
|--------------------|--------|---------------|
| └ ○ /              | 2.5 kB | 105 kB        |
| └ ○ /admin         | 3.8 kB | 106 kB        |
| └ ○ /auth/callback | 1.2 kB | 100 kB        |
| └ ○ /auth/login    | 1.8 kB | 102 kB        |
| └ ○ /auth/logout   | 143 B  | 97.8 kB       |
| └ ○ /dashboard     | 4.1 kB | 108 kB        |
| └ ○ /drill         | 6.2 kB | 112 kB        |

If you see TypeScript or ESLint errors in the build output, fix them before pushing. The CI pipeline on Vercel runs the same build and will fail on the same errors.

---

## 10. Smoke Test the App

Walk through this checklist after getting the server running. It covers the full user journey and exercises every major system.

### Anonymous drill flow

- ☐ Navigate to `http://localhost:3000` — homepage loads with today's topic and streak = 0
- ☐ Click **Start Today's Drill** — Lesson Card renders with topic title, lesson text, and "Start Drill" button
- ☐ Click **Start Drill** — first question renders
- ☐ Answer all 10 questions — each answer reveals the correct option and a coaching explanation
- ☐ Click **See Results** — score screen renders with coaching message
- ☐ Expand **Compare Perspectives** — key idea bullets and model response appear
- ☐ Write text in the journal textarea and click **Save This Insight** — confirmation message appears
- ☐ Click **Review Lesson** — all 10 questions render with expandable coaching notes

### Dashboard (anonymous)

- ☐ Navigate to `/dashboard` — streak badge, stat cards, and session history render
- ☐ Session from the drill above appears in session history

### Authentication

- ☐ Navigate to `/auth/login` — email input renders
- ☐ Enter your ADMIN\_EMAIL address and click **Send Magic Link** — confirmation message appears
- ☐ Open the magic link email and click the link — browser redirects to `/dashboard` signed in
- ☐ Nav shows your email address and a **Sign out** button

### Admin dashboard

- ☐ Navigate to `/admin` while signed in as ADMIN\_EMAIL — instructor dashboard renders with metrics
- ☐ Navigate to `/admin` in an incognito window (not signed in) — redirects to `/auth/login`

### Sign out

- ☐ Click **Sign out** — redirects to homepage, nav shows "Sign in" again

If any step fails, see Section 15 (Common Issues and Fixes) before debugging further.

---

## 11. Project Orientation

After the smoke test you have a working app. Now orient yourself to the codebase.

## The files you will touch most often

| File / folder                | What it is                                                                               |
|------------------------------|------------------------------------------------------------------------------------------|
| app/                         | All pages and route handlers (Next.js App Router)                                        |
| components/                  | All React components, organised by feature (drill/, dashboard/, admin/)                  |
| data/questions/beginner.json | The question bank — 72 questions with coaching notes                                     |
| lib/questions/selector.ts    | Picks 10 questions for a drill session                                                   |
| lib/questions/topics.ts      | Topic metadata, lesson content, reflection prompts, Compare Perspectives content         |
| lib/utils/streak.ts          | Computes streak from session dates                                                       |
| lib/utils/progress.ts        | localStorage read/write for anonymous users                                              |
| lib/supabase/                | Three Supabase clients: client.ts (browser), server.ts (server), admin.ts (service role) |
| proxy.ts                     | Session-refresh middleware (runs on every request)                                       |
| types/index.ts               | Shared TypeScript types: Question, DrillSession, UserProgress                            |

## The dual data path

The single most important architectural concept in this codebase is the dual data path:

- **Anonymous users** — all progress is stored in localStorage under the keys `kmr-progress` (sessions) and `kmr-journal` (journal entries). `lib/utils/progress.ts` handles reads and writes.
- **Signed-in users** — all progress is stored in Supabase. The dashboard server component fetches it and passes it as props to `DashboardView`.
- **DashboardView.tsx** merges these two sources. It receives server-fetched data as props and reads localStorage on mount. It handles both cases transparently without different UI layouts.

When adding any feature that involves persisting data, ask yourself: does this need to work for anonymous users, signed-in users, or both? The answer determines which path you touch.

## The drill state machine

`components/drill/DrillClient.tsx` is the core of the app. It is a client-side state machine with five phases:



| Phase     | What renders                                                   |
|-----------|----------------------------------------------------------------|
| lesson    | LessonCard — topic overview before the drill starts            |
| answering | QuestionCard + AnswerOption components — active question       |
| revealed  | Answer shown with Explanation — learner sees correct/incorrect |
| completed | ScoreDisplay — score, reflection, Learning Journal             |
| review    | LessonReview — all 10 questions with per-option coaching       |

On completion, DrillClient fires a POST /api/scores request (fire-and-forget) to sync the session to Supabase for signed-in users.

### The question bank shape

Each question in data/questions/beginner.json has this shape:

```
{
  id: string,           // stable identifier – referenced in drill_question_answers
  type: "multiple_choice" | "true_false" | "scenario",
  category: string,     // one of 6 slugs: debits_credits, accounts, etc.
  level: "beginner",
  text: string,
  options: string[],    // 2 options for true/false, 4 for everything else
  correctIndex: number, // 0-based index into options[]
  explanation: string,  // coaching explanation shown after the answer
  optionExplanations: string[] // one coaching note per option (why it's right or wrong)
}
```

question.id is stored in drill\_question\_answers.question\_id. **Never change or reuse an existing id value** — doing so corrupts the analytics history for any user who has answered that question.

## 12. Next.js 16 — Breaking Changes to Know

This project uses Next.js 16, which has several breaking changes from earlier versions. If your knowledge of Next.js comes from version 13 or 14 training data, read node\_modules/next/dist/docs/ before writing any code. The most common surprises are documented here.

### Middleware is proxy.ts, not middleware.ts

Next.js 16 renames the middleware entry point from middleware.ts to proxy.ts. The file at the root of this project is proxy.ts. Do not create a middleware.ts file — it will

be silently ignored.

```
// proxy.ts – runs on every request, refreshes the Supabase session  
export default async function proxy(request: NextRequest) { ... }  
export const config = { matcher: [...] }
```

## Route params are async (Promise)

In Next.js 16, dynamic route params are wrapped in a Promise. You must await them before use.

```
// Next.js 16 – correct  
export default async function Page({ params }: { params: Promise<{ id: string }> }) {  
  const { id } = await params  
}  
  
// Earlier Next.js – no longer works  
export default function Page({ params }: { params: { id: string } }) {  
  const { id } = params // will throw in Next.js 16  
}
```

## cookies() is async

The cookies() function from next/headers is now async.

```
// Next.js 16 – correct  
const cookieStore = await cookies()  
  
// Earlier Next.js – no longer works  
const cookieStore = cookies()
```

lib/supabase/server.ts already handles this correctly — follow its pattern when creating server-side Supabase clients.

## Tailwind v4 — no @tailwind directives

Tailwind v4 uses a single import instead of the three-directive pattern.

```
/* app/globals.css – Tailwind v4 (correct) */  
@import "tailwindcss";  
  
/* Do NOT use the old pattern */  
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

There is no tailwind.config.ts file in this project. Tailwind v4 auto-detects content files from the project structure.

## **No separate `eslint.config.js` setup needed**

ESLint 9 with flat config is used. The config lives in `eslint.config.mjs`. Run `npm run lint` to check.

---

## **13. Working in This Codebase**

### **Before starting any task**

1. `git pull origin main` to ensure you are on the latest code
2. Read the relevant section of the Technical Blueprint for the area you are changing
3. For significant changes, update the relevant document in `docs/` before writing code

### **Running a build before pushing**

Always run a production build before pushing to main:

```
npm run build
```

This catches TypeScript errors and missing imports that the dev server (which has more lenient checking) may not surface.

### **Linting**

```
npm run lint
```

Fix all lint errors before pushing. The Vercel build will fail on lint errors.

### **Adding questions to the question bank**

1. Open `data/questions/beginner.json`
2. Add new questions to the array following the shape documented in Section 11
3. Assign a new unique id (use a descriptive slug: `debits_credits_q13`)
4. Ensure every question has a non-empty explanation and a non-empty `option-Explanations` entry for every option
5. Run the build and check for TypeScript errors
6. Manually play through a drill to verify the new questions render and the coaching notes display correctly

### **Adding a new topic category**

Topic metadata — lesson text, reflection prompts, Compare Perspectives content — lives in `lib/questions/topics.ts`. To add a new category:

1. Add the new category slug to the `topics` record in `topics.ts`
2. Add questions tagged with the new category slug to the question bank

3. Update the rotation formula in `lib/questions/selector.ts` if the total category count changes
4. Update the topic rotation table in the Technical Blueprint

## Changing the database schema

If your change requires a new table or column:

1. Write the SQL as a new migration file: `supabase/migrations/003_your_feature.sql`
2. Use `IF NOT EXISTS` / `IF NOT EXISTS` guards so the migration is safe to re-run
3. Apply it manually in the Supabase SQL Editor for your local dev project
4. Apply it to the production project after the code change is deployed
5. Update the database schema section of the Technical Blueprint

Never modify the existing migration files — they are the historical record of how the schema evolved.

---

## 14. Deployment

Every push to main triggers an automatic Vercel deployment. There is no staging environment in v1.0.

### First-time setup

If you are deploying to a new Vercel project:

1. Push the repository to GitHub
2. Import the repository at [vercel.com/new](https://vercel.com/new)
3. Vercel auto-detects Next.js — no build configuration needed
4. Go to **Settings** → **Environment Variables** and add all four variables from `.env.local`:
  - `NEXT_PUBLIC_SUPABASE_URL`
  - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
  - `SUPABASE_SERVICE_ROLE_KEY`
  - `ADMIN_EMAIL`
5. Set scope to **Production** (and optionally Preview)
6. Trigger a deploy

### After deploying, update Supabase Auth URLs

Go to your Supabase project → **Authentication** → **URL Configuration**:

- Set **Site URL** to `https://your-app.vercel.app`
- Add `https://your-app.vercel.app/auth/callback` to **Redirect URLs**

Magic links will not work in production until this step is done.

## Triggering a redeploy without a code change

To pick up new environment variables:

```
git commit --allow-empty -m "chore: trigger redeploy" && git push origin main
```

Or use the Vercel dashboard: **Deployments** → ⋯ → **Redeploy**.

## Checking function logs

Server-side errors (route handlers, server components) appear in Vercel → **Logs** → **Functions**. There is no error monitoring service in v1.0 — check here when something appears broken in production.

---

## 15. Common Issues and Fixes

### Magic link arrives but clicking it gives an error

**Symptom:** The magic link redirects to `/auth/callback` but the page shows an error or redirects back to login.

**Cause:** The redirect URL in Supabase Auth is not whitelisted.

**Fix:** Go to **Authentication** → **URL Configuration** in Supabase and confirm that the URL you clicked the link from (including scheme and port, e.g. `http://localhost:3000/auth/callback`) is in the Redirect URLs list.

---

### `/admin` redirects to `/auth/login` even when signed in

**Symptom:** You are signed in and navigating to `/admin` sends you to the login page.

**Cause:** The session is not being read server-side, usually because `.env.local` is missing or `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` are incorrect.

**Fix:** Verify your `.env.local` values match the Supabase project you configured. Restart the dev server after changing `.env.local` — Next.js does not hot-reload env changes.

---

### `/admin` shows “Access Denied” with your email

**Symptom:** You are signed in, the page renders, and it says “Access Denied” with your email shown.

**Cause:** The `ADMIN_EMAIL` env var does not match the email you signed in with. This is by design — the page shows your email to help diagnose the mismatch.

**Fix:** Update `ADMIN_EMAIL` in `.env.local` to match your signed-in email exactly (case-sensitive), then restart the dev server.

---

## Dashboard shows no data after completing a drill

**Symptom:** You complete a drill, navigate to /dashboard, and see zeroed stats.

**Cause (anonymous users):** localStorage is blocked or unavailable. Check browser privacy settings — private browsing modes in some browsers disable localStorage.

**Cause (signed-in users):** The drill\_sessions table may not exist yet. Confirm you ran both migrations (Section 7) and that the drill\_sessions table appears in the Supabase Table Editor.

---

## Build fails with “Module not found” for Supabase imports

**Symptom:** npm run build fails with an error like Cannot find module '@supabase/ssr'.

**Fix:**

```
rm -rf node_modules
npm install
```

---

## Tailwind styles not applying

**Symptom:** Pages render unstyled or partially styled.

**Cause:** Tailwind v4 auto-discovers class usage from source files. If you move files outside the standard app/, components/, or lib/ folders, Tailwind may not scan them.

**Fix:** Ensure new files with Tailwind classes are inside one of those directories. If adding a new top-level folder, add it to the PostCSS config if necessary.

---

## cookies() throws “was called outside a request scope”

**Symptom:** A server component or route handler throws this error at runtime.

**Cause:** The Supabase server client is being initialised at module scope (outside a function body) rather than inside a request handler.

**Fix:** Follow the pattern in lib/supabase/server.ts — always create the server client inside the function that handles the request, never at the top level of a module.

---

*Keep Me Ready — Practice makes permanent.*